CS-4063: Natural Language Processing

Assignment 3 — Transformers + RAG

Name: Amaim Anwar
Section: DS-6A
Roll-no: 23i-2614

1. Overall System Design and Methodology

This assignment implements a three-stage Retrieval-Augmented Generation (RAG) pipeline for Amazon product review understanding and explanation generation. The pipeline is built entirely from scratch in PyTorch, with no use of pretrained models, HuggingFace, or any forbidden built-ins (nn.Transformer, nn.MultiheadAttention, nn.TransformerEncoder). The three stages correspond to the three assignment parts:

- Part A — Encoder-only Transformer: reads a review and produces a sentiment label, a verbosity label, and a fixed-dimensional embedding vector.
- Part B — Retrieval Module: uses the encoder embeddings to find the top-k most similar training reviews for any query review at inference time.
- Part C — Decoder-only Transformer: takes the original review, both predicted labels, and the retrieved context, then generates a one-to-two sentence natural language explanation of the predicted sentiment.

The three stages are designed to work as a cohesive pipeline: the encoder produces embeddings that feed the retrieval index, and the retrieved context enriches the decoder input. The dataset used is the Amazon Reviews dataset across three product categories — Electronics, Sports, and Beauty — yielding 39,000 reviews in total.

2. Preprocessing Pipeline

2.1 Dataset Construction

Three product categories were selected: Electronics, Sports, and Beauty. Exactly 13,000 reviews were sampled from each category, giving 39,000 samples in total. Each sample contains the review text and its corresponding 1–5 star rating. Reviews with fewer than five words were discarded. The dataset was

stratified by sentiment label and split into 70% training (27,299), 15% validation (5,849), and 15% test (5,852).

Split	Samples	Negative	Neutral / Positive
Train	27,299	2,564	2,349 / 22,386
Validation	5,849	549	504 / 4,796
Test	5,852	551	504 / 4,797

2.2 Text Cleaning

All preprocessing steps are implemented from scratch. The cleaning pipeline applies the following steps in order: (1) conversion to lowercase; (2) removal of HTML tags using regex; (3) removal of URLs; (4) retention of alphabetic characters only, removing digits and special characters; and (5) collapsing of multiple whitespace characters into a single space. These steps ensure that the encoder learns from normalised, noise-free text.

2.3 Tokenization and Vocabulary

A simple whitespace tokenizer splits each cleaned review into a list of word tokens. The vocabulary is constructed exclusively from the training set to prevent data leakage. Only words appearing at least 3 times are retained, and the vocabulary is capped at 30,000 entries. Four special tokens are reserved: <PAD> (index 0), <UNK> (index 1), <BOS> (index 2), and <EOS> (index 3). The final vocabulary contains 16,919 tokens. Unknown tokens account for only 1.0% of training tokens and 1.3% of test tokens, indicating strong vocabulary coverage.

2.4 Encoding and Padding

Each cleaned review is converted to a sequence of token indices. A fixed maximum sequence length of 128 tokens is applied: sequences longer than 128 tokens are truncated, and shorter sequences are padded with the PAD token. Approximately 21% of training reviews required truncation. The mean token length before truncation is 68.8 tokens, with a median of 58 tokens.

2.5 Label Definitions

Two label sets are defined. For sentiment, the assignment specification requires: ratings 1-2 map to Negative (label 0), rating 3 to Neutral (label 1), and ratings 4-5 to Positive (label 2). The dataset is heavily skewed towards Positive (82.0%), reflecting typical Amazon review patterns. For the second task, verbosity is defined as a proxy for reviewer engagement: reviews with fewer than 50 words are Short (label 0, 44.2%), 50-150 words are Medium (label 1, 40.0%), and over 150 words are Long (label 2, 15.7%). Verbosity can be learned from linguistic patterns in the text.

3. Part A — Encoder Model for Understanding

3.1 Architecture

An encoder-only Transformer is implemented from scratch. The architecture follows the standard Transformer encoder design with Pre-Layer Normalisation residual connections. A learned [CLS] token is prepended to every input sequence; its final hidden state serves as the review embedding used for both classification and retrieval.

Hyperparameter	Value
d_model (embedding dim)	128
Number of attention heads	4
Number of encoder layers	3
Feed-forward dim (d_ff)	256
Dropout	0.2
Max sequence length	128
Vocabulary size	16,919
Total parameters	2,579,590

3.2 Component Design

Scaled dot-product attention is implemented as Attention(Q,K,V) = softmax(QK^T / sqrt(dk)) * V with an optional padding mask that sets pad positions to negative infinity before the softmax. Multi-head self-attention uses four independent heads, each with separate W_Q, W_K, and W_V projection matrices of dimension 128 x 32. The outputs are concatenated and projected through a shared W_O matrix. The position-wise feed-forward network uses two linear layers with a ReLU activation and

inner dimension 256. Sinusoidal positional encoding is stored as a fixed, non-learnable buffer and added to the token embeddings before the first encoder block.

3.3 Multi-Task Heads

Both classification tasks share the CLS token representation as input. Two separate MLP heads (128 -> 64 -> ReLU -> Dropout -> 3) are attached: one for sentiment and one for verbosity. The combined training loss is a weighted sum: L = 0.5 * CrossEntropy(sentiment) + 0.5 * CrossEntropy(verbosity). Equal weighting was chosen after experimentation, as sentiment and verbosity are complementary tasks of similar difficulty.

3.4 Training

The model is trained for 10 epochs with the Adam optimizer (learning rate 0.0003) and a cosine annealing learning rate schedule. Gradient clipping at 1.0 is applied for stability. The model achieving the lowest validation loss is saved as the best checkpoint.

3.5 Results

Task	Accuracy	Macro-F1	Best Val Loss
Sentiment (3-class)	0.8383	0.4196	0.3224
Verbosity (3-class)	~0.93	0.9124	0.3224

The high accuracy (83.8%) on sentiment is driven by the class imbalance: 82% of the dataset is Positive, so macro-F1 (0.42) more honestly reflects performance. The Neutral class is the hardest to classify due to the ambiguity of 3-star reviews. Verbosity is a much easier task and achieves a macro-F1 of 0.91, validating the design choice of using word-count-based labels as the auxiliary signal.

4. Part B — Retrieval Module

4.1 Embedding Storage and Indexing

After training, the encoder is run over all 27,299 training reviews in inference mode (no gradient). The CLS token embedding (dimension 128) of each review is extracted and stored as a numpy array of shape (27,299, 128). This index is saved to disk as results/train_embeddings.pkl alongside aligned metadata (text, rating, category, sentiment, verbosity).

4.2 Query Construction and Similarity Search

At inference time, a test review is passed through the encoder to produce its CLS embedding (query vector). All stored training embeddings are L2-normalised and the query vector is similarly normalised, so cosine similarity reduces to a dot product. The top-k training reviews with the highest cosine similarity scores are retrieved as the context. The value of k is set to 5 by default and is configurable. Cosine similarity was chosen because it measures directional alignment between embedding vectors, making it invariant to vector magnitude and more robust than Euclidean distance for high-dimensional semantic representations.

4.3 Retrieval Quality Analysis

k	Precision@k	Mean Cosine Sim (top-k)	Mean Cosine Sim (top-1)
1	0.8469	0.9973	0.9973
3	0.8460	0.9970	0.9973
5	0.8479	0.9968	0.9973
10	0.8466	0.9965	0.9973

Precision k measures the fraction of retrieved reviews that share the same predicted sentiment as the query. Values around 84-85% across all k values confirm that the encoder embeddings encode sentiment-discriminative structure. The very high cosine similarity scores (0.997+) suggest that the embedding space is tightly clustered. k=5 was chosen as it provides the best precision with enough retrieved diversity to ground the decoder.

A limitation of this approach is that the retrieval is entirely based on encoder embeddings that may cluster by verbosity or category rather than by nuanced semantic meaning. Future improvements could include using separate embedding heads per task or employing approximate nearest-neighbour search (FAISS) for efficiency at scale.

5. Part C — Decoder Model for Explanation Generation

5.1 Input Template

The decoder input is a structured sequence combining all four required elements: (1) the original review text (cleaned), (2) the predicted sentiment label, (3) the predicted verbosity label, and (4) the top-3 retrieved review snippets (truncated to 30 tokens each). The template is:

review [review_text] sentiment [label] verbosity [label] context1 [snippet1] context2 [snippet2] context3 [snippet3] <BOS> [explanation] <EOS>

The reference explanation follows a fixed template: "this review is [sentiment] because the reviewer expresses [attitude]. similar reviews also show [sentiment] sentiment as [N] out of 5 retrieved examples share comparable language." The model is supervised only on the explanation tokens after <BOS>; the prompt portion does not contribute to the loss.

5.2 Decoder Architecture

A decoder-only Transformer is implemented from scratch. The key constraint is the causal (lower-triangular) attention mask that prevents any position from attending to future tokens, enabling autoregressive generation. Each decoder block uses Pre-Layer Normalisation with the structure: x = x + Dropout(MaskedSelfAttention(LN(x))), followed by x = x + Dropout(FFN(LN(x))). Weight tying between the token embedding matrix and the output projection layer reduces parameters and is a standard technique for language models.

Hyperparameter	Value
d_model	128
Attention heads	4 (causal mask)
Decoder layers	3
d_ff	256
Dropout	0.2
Total parameters	2,562,176
Weight tying	Yes

5.3 Training

The decoder is trained for 10 epochs with the Adam optimizer (learning rate 3e-4) and a cosine annealing schedule. The language modelling objective (cross-entropy loss over explanation tokens

only) is minimised. Gradient clipping at 1.0 is applied. Training on GPU (T4 via Google Colab) takes approximately 35 seconds per epoch.

5.4 Quantitative Evaluation — Perplexity

System	Test Loss	Test PPL	Val PPL (best)
Full RAG (with retrieval)	0.0212	1.02	1.02
Baseline (no retrieval)	0.0096	1.01	1.01

5.5 Qualitative Evaluation

Five generated explanations were examined. The decoder successfully reproduces the template structure in most cases, generating coherent sentiment-aligned explanations. For Positive reviews, the model reliably produces phrases such as "the reviewer expresses satisfaction". For Negative reviews, it generates "the reviewer expresses dissatisfaction and frustration". Neutral cases are the weakest, reflecting the same ambiguity seen in the encoder. The generated text is grammatically well-formed but occasionally omits specific retrieved evidence words. This is expected given the small model size and the templated nature of the targets.

6. Hyperparameter Tuning Log

6.1 Encoder Tuning

Config	d_model	Heads	Layers	Val Sent Acc	Notes
1 (baseline)	64	2	2	~0.71	Too small
2	128	4	2	~0.78	Shallow
3 (chosen)	128	4	3	~0.83	Best tradeoff
4	256	4	4	~0.82	3x slower

Config 3 with d_model=128, 4 heads, and 3 layers was selected as the best encoder configuration. Config 4 added only marginal accuracy improvement (+~0.5%) while tripling training time on CPU, making it impractical for Colab.

6.2 Decoder Tuning

Config	d_model	Heads	Layers	Val PPL	Notes
1	64	2	2	High	Underfits
2	128	4	2	1.03	Decent
3 (chosen)	128	4	3	1.02	Best, stable
4 (high lr)	128	4	3	Unstable	lr=1e-3 spikes
5	256	8	4	1.02	3x slower

7. RAG Ablation Study

To assess the contribution of the retrieval module, a baseline decoder is trained without any retrieved context. The baseline receives only the review text, sentiment label, and verbosity label as input. Both systems are trained identically (same architecture, optimizer, schedule, and number of epochs) on the same data splits.

System	Test Loss	Test PPL	PPL Change
Full RAG (k=5 retrieved)	0.0212	1.02	—
Baseline (no retrieval)	0.0096	1.01	-1.16%

The full RAG system has a slightly higher perplexity (1.02) than the baseline (1.01). The margin is small (-1.16%) and is expected given that the target explanations are templated sequences. Because the reference explanation is generated deterministically from the predicted sentiment label, a decoder that has memorised the template achieves low perplexity even without retrieved context. The retrieval module primarily benefits the model when the target explanations contain specific retrieved content — for example, when the explanation counts the number of retrieved examples that agree in sentiment. The side-by-side qualitative comparison confirms that both systems produce similar outputs on Positive reviews, but the RAG system produces slightly more grounded language for Negative and Neutral reviews by conditioning on retrieved negative-sentiment neighbours. Future work with more diverse, human-written reference explanations would better demonstrate the retrieval benefit.

8. Design Decision Justifications

- CLS token: A learned CLS token prepended to every input sequence provides a single fixed-dimensional summary vector without averaging or pooling, which is the standard approach in BERT-style encoders.
- Pre-Layer Normalisation: Pre-LN residual connections (normalise before the sub-layer) are more stable to train than Post-LN, especially for smaller models, and typically converge faster.
- Cosine similarity for retrieval: Invariant to vector norm, robust to high-dimensional spaces, and computationally efficient via L2-normalised dot product.
- k=5 for retrieval: Provides enough context diversity while keeping the decoder input sequence manageable. Precision k remains stable from k=1 to k=10, so k=5 is a good default.
- Verbosity as the second task: Verifiable purely from token count, provides a complementary training signal to sentiment, and does not leak sentiment information into the auxiliary label.
- Causal mask in decoder: A lower-triangular boolean mask is applied before the softmax in the decoder's self-attention to prevent future-token attending, which is essential for autoregressive generation.
- Weight tying: The output projection matrix is shared with the token embedding matrix, reducing parameters and encouraging consistent token representations between input and output spaces.

9. Conclusion

This assignment successfully implements a complete RAG pipeline from scratch using PyTorch, comprising an encoder-only Transformer for multi-task review understanding, a cosine-similarity retrieval module, and a decoder-only Transformer for explanation generation. The encoder achieves 83.8% accuracy on sentiment classification and 91.2% macro-F1 on verbosity prediction. The retrieval module achieves Precision@5 of 84.8% with mean cosine similarity of 0.997, demonstrating that the encoder embeddings capture semantically meaningful structure. The decoder achieves test perplexity of 1.02 for the full RAG system. The ablation study shows that the retrieval contribution is modest under templated targets, but provides qualitative grounding improvements for minority-class reviews. All components are implemented strictly from scratch without any pretrained models or forbidden built-ins.